

Discussion-6 Strengths and limitations of sequence diagrams

Discussion Topic:

What are the strengths and limitations of sequence diagrams? Provide specific scenarios in your post. In your peer responses, please provide additional examples of the uses of sequence diagrams.

My Post:

Hello Class,

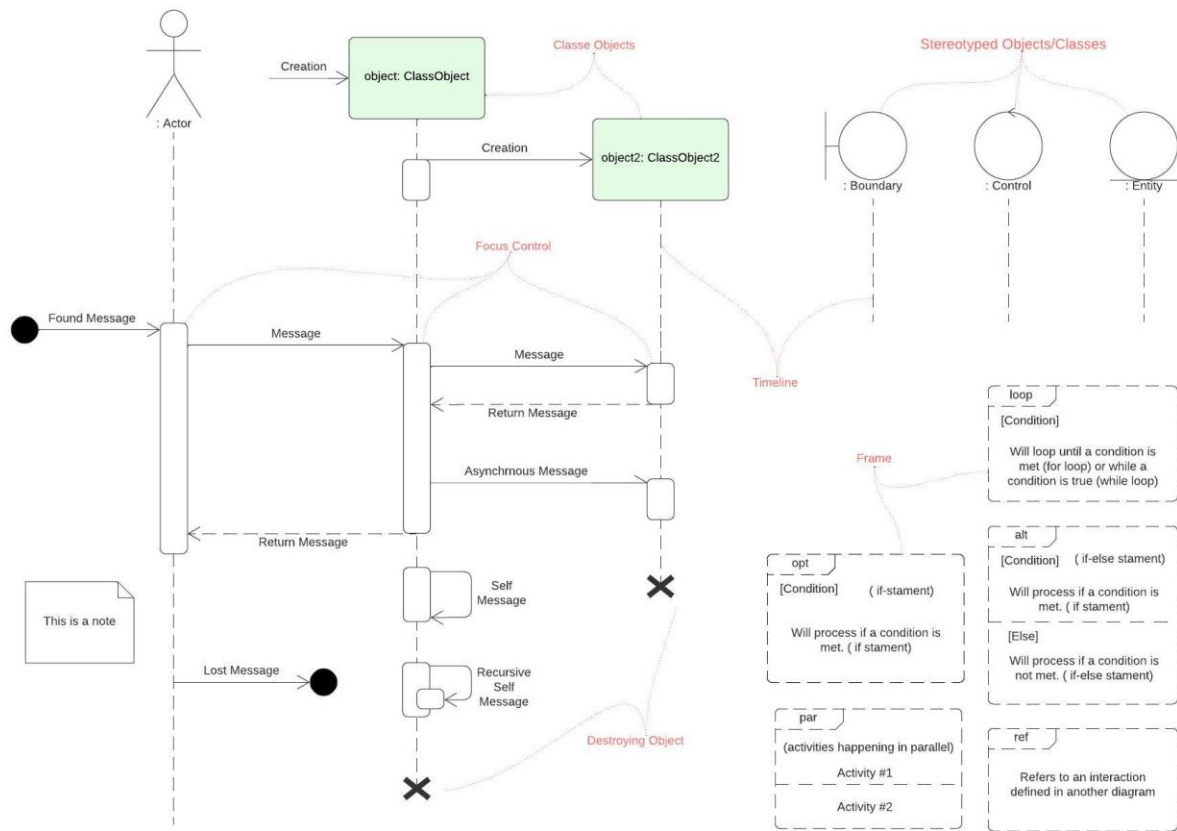
Unified Modeling Language (UML) sequence diagrams illustrate the sequence of messages in an interaction (IBM, 2021). They are a form of interaction diagram used in Software Engineering (SE) to model the interactions between objects in a single use case. This post provides an overview of the UML sequence diagrams and their strengths and limitations in software development.

UML Sequence Diagrams Overview

UML sequence diagrams provide a dynamic illustration of object interactions within a system. In SE, they are usually based on UML class diagrams and are used primarily to show the interactions between objects (classes) in the chronological order in which those interactions occur. In the problem space, they are used to model the behavior of systems from the actors' (users) perspective (Unhelkar, 2018). In solution space, the diagrams are more detailed and used to illustrate objects' interactions and their messages in detail such as the sequence of those messages, a parameter list within messages, and the return values of the messages. The figure below illustrates the different elements that can be found in a UML sequence diagram.

Figure 1

UML Sequence Diagram Notation

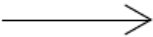
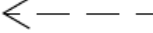
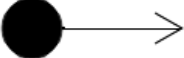
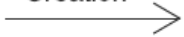


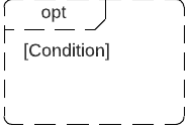
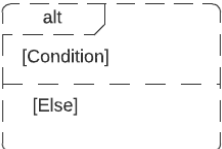
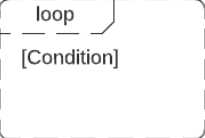
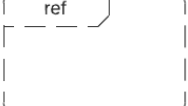
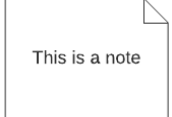
Note: made with the Lucidchart app.

The table below describes some of the main notation elements that can be found in UML sequence diagrams.

Table 1
UML Sequence Diagram Notation Descriptions

Notation	Description
Actor 	Represents a user (outside actor) or external system.
Lifeline or Timeline 	Represents a participant lifeline in the system

Boundary 	Stereotyped object/class representing a system boundary element, like UI screens or database gateways.
Control 	Stereotyped object/class representing a controlling entity or manager.
Entity 	A stereotyped object/class usually represents system data or a data object.
Focus or Activity Bar 	A thin rectangle on a lifeline that shows the period during which an object is active or performing an action.
Message 	Synchronous messages that represent inquirer messages between objects (e.g. <code>getAttribute()</code>) - The sender waits for confirmation or return data from the receiver to continue processing.
Asynchronous Message 	An asynchronous message, the sender does not wait for confirmation or return data from the receiver to continue processing.
Return Message 	A dashed arrow with an open arrowhead. Indicates the return of control to the message caller.
Found Message 	The message is outside of the scope of the system description; for example, a message that originated outside of the system.
Lost Message 	A message that does not have a receiver or the receiver is unknown; for example, a message sent to an outside unknown system.
Creation Arrow Creation 	Represents the point in a system timeline when a particular object was created and may also indicate who/what created it.
Participant Destruction 	An "X" at the end of a lifeline. Indicates that the participant is destroyed.

Option Frame 	"if-then" logic.
Alternatives Frame 	"if-then-else" logic.
Loop Frame 	Represents the logic of a “for-loop” or “while-loop”
Parrelle Frame 	Represents activities happening in parallel (at the same time).
Reference Frame 	Allows reusing part of one sequence diagram in another. Represented by a frame that refers to another diagram.
Comments 	Comments are used to add explanatory notes to the diagram.

Note: a stereotyped object or class is an object with a specific purpose or functionality defined by a stereotype (e.g. boundary, control, and entity). The data was collected from several sources (Visual-Paradigm n.d.; Unhelkar, 2018; rmb1905, 2009).

Strengths and Limitations of UML Sequence Diagrams

UML sequence diagrams are excellent for visualizing the dynamic behavior of a system, particularly the interactions between objects or components over time. They can be used for analyzing and designing object interactions within a system, they are especially helpful in identifying missing elements in class diagrams and they can be used for documentation. See Table for a description of diagram strengths and limitations. For example, when designing a shopping cart system, they are especially useful at showing in

detail the user (actor) interacting with the shopping cart and checkout system (objects). A sequence diagram can clearly depict the sequence of messages, for instance, addItem, viewCart, initiateCheckout, processPayment, etc., that is the exchanges between the user, the shopping cart, and the checkout system.

However, sequence diagrams have also their limitations, they are not well-suited for representing highly concurrent or parallel processes. Even though UML 2 implemented the parallel frame element, the dynamic nature of the diagrams makes it difficult to depict all the concurrent and parallel interactions that may exist in a complex system. In other words, trying to illustrate many concurrent and parallel interactions in a single diagram would make it extremely large and cluttered and ultimately it may require the generation of several additional sequence diagrams. Activity diagrams are better suited for such scenarios as they support parallel flows by using fork and join nodes.

Table 2
Strengths and Limitations of UML Sequence Diagrams

	Strengths	Limitations
Illustration	- Visually represent user-described examples, scenarios, or storyboards, especially useful for mobile app development - Depict a sequence of messages between objects within a specific timeframe, showing preconditions - Provide a "snapshot" of system activity during a specific time period.	- Incomplete, showing only snapshots; unsuitable for depicting entire processes or flows - Cannot easily represent conditional logic (if-then-else) or loops (for-next). - Represent only single threads; multiple threads require separate, unrelated diagrams - Representing entire sequence in one diagram can lead to confusion; only model appropriate subsets.
Object - Class	- Help identify missing objects that can be added as classes in a class diagram - Help identify missing operations/methods within classes, which can be added to a class diagram - Can display multiple objects of the same class ("instance diagrams").	
Design	- Methods with signatures provide detailed design information for designers and programmers - Mapping between sequence and class diagrams improves class diagrams. - Showing object destruction ensures proper deletion and reduces memory clutter - Can show the same message being passed between objects multiple times, emphasizing dynamic behavior.	- The number of sequence diagrams needed is unclear and relies on the designer's judgment - Misaligned usage between system designers and business analysts can be

Note: data from "Chapter-12 Interaction Modeling with Sequence Diagrams" by Unhelkar (2018).

To summarize UML sequence diagrams are a useful tool for illustrating dynamic interactions of objects within a system. In SE, they are usually based on UML class diagrams and they are in the problem space to model the behavior of systems from the user perspective, in solution space, the diagrams are more detailed and used to illustrate objects' interactions such as the sequence of the message, a parameter

list within messages, and the return values of the messages. They help identify missing elements, detail object interactions, and model object deletion. However, the diagrams have limitations such as representing single flows or threads, they do not illustrate or model well parallelism, conditional logic, or loops, and using a single sequence diagram for an entire complex system can lead to confusion and errors. Nonetheless, they are an indispensable and powerful tool in the software engineer's toolkit, they are essential for designing, analyzing, and documenting interactions within a system.

-Alex

References:

IBM (2021, March 5). Sequence diagrams. *Rational Software Modeler*. IBM Documentation. <https://www.ibm.com/docs/en/rsm/7.5.0?topic=uml-sequence-diagrams>

rmb1905. (2009, March 10). 10.09_Fragments - parallels [Video]. YouTube. <https://www.youtube.com/watch?v=mtVVf1mhYKk>

UML Diagrams Org. (n.d.). *UML sequence diagrams*. UML Diagrams Org. <https://www.uml-diagrams.org/sequence-diagrams.html#execution>

Unhelkar, B. (2018). Chapter 12 — Interaction modeling with sequence diagrams. *Software engineering with UML*. CRC Press. ISBN 9781138297432

Visual Paradigm (n.d.). *Sequence diagram*. Visual Paradigm. <https://www.visual-paradigm.com/VPGallery/diagrams/Sequence.html>